UNIVERSIDAD NACIONAL JORGE BASADRE GROHMANN

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL EN INFORMÁTICA Y SISTEMAS



Comparación empírica entre los algoritmos de ordenación por inserción y Quicksort con pivote mediana de tres

Empirical comparison between the insertion sort and Quicksort (with median of three pivot) algorithms

Estudiante:

Natalia Lizeth Huanca Quispe

Docente:

Dr. Hugo Manul Barraza Vizcarra

TACNA – PERÚ

2026

Resumen

Se comparó el ordenamiento por inserción con Quicksort de Hoare y pivote mediana de tres en cinco patrones de entrada y dos tamaños (n = 1 000 y 10 000). Para cada combinación de tamaño, patrón y algoritmo se realizaron 30 repeticiones. Ambos métodos recibieron copias idénticas de cada instancia. Se midieron tiempo con std::chrono::steady_clock, comparaciones, intercambios y profundidad máxima de recursión. La generación quedó fuera del tiempo medido. A n = 10 000, Quicksort fue entre 71 y 2 584 veces más rápido que inserción en cuatro patrones; en orden ascendente, inserción tuvo menor tiempo medio (0,032 frente a 0,279 ms). El orden previo afectó de forma marcada a inserción, mientras que Quicksort mantuvo tiempos inferiores a 1,20 ms en los cinco patrones. Los datos y semillas por corrida se guardaron en CSV para permitir la reproducción del análisis.

Palabras clave: ordenamiento, Quicksort, complejidad algorítmica, análisis empírico, reproducibilidad

Abstract

This study compares insertion sort with Hoare's Quicksort using a median-of-three pivot across five input patterns and two input sizes (n = 1,000 and 10,000). Each algorithm, pattern, and size condition was repeated 30 times. Both algorithms received identical copies of each generated instance. We measured runtime with std::chrono::steady_clock, comparisons, swaps, and maximum recursion depth. Input generation was excluded from timed sections. At n = 10,000, Quicksort was 71 to 2,584 times faster than insertion sort on four patterns; insertion sort had a lower mean time on ascending input (0.032 versus 0.279 ms). Input order strongly affected insertion sort, whereas Quicksort remained below 1.20 ms across all five patterns. Per-run seeds and raw observations were recorded in CSV to support reproducibility.

Keywords: sorting, Quicksort, algorithmic complexity, empirical analysis, reproducibility

Introducción

La elección de un método de ordenamiento depende del tamaño y de la estructura de los datos. La inserción tiene un costo cuadrático en promedio y en el peor caso, pero puede ejecutarse en tiempo lineal cuando la entrada ya está ordenada. Quicksort divide la secuencia mediante particiones y suele alcanzar O(n log n) en promedio, aunque su peor caso es O(n²) (Cormen et al., 2022). Hoare presentó Quicksort como un método interno de ordenamiento, y análisis posteriores describen cómo la selección del pivote influye en su rendimiento práctico (Hoare, 1962; Sedgewick, 1978).

Este estudio pregunta cómo cambian el tiempo y el número de operaciones de inserción y Quicksort al variar el tamaño y el patrón de entrada. La hipótesis descriptiva sostiene que inserción será competitiva en datos ascendentes o casi ordenados, mientras que Quicksort obtendrá menor tiempo en entradas aleatorias, descendentes y con alta repetición, especialmente al aumentar n. El objetivo es contrastar esa expectativa con observaciones reproducibles del programa implementado.

Método

Algoritmos y complejidad

La Inserción desplaza los elementos mediante intercambios adyacentes hasta ubicar cada clave. Mantiene el orden relativo de valores iguales y usa memoria auxiliar constante. Su mejor caso es O(n); el promedio y el peor caso son O(n²). Quicksort aplica la partición de Hoare; elige el pivote como la mediana de los elementos primero, central y último de cada subarreglo. Es inestable, usa memoria de pila O(log n) en promedio y puede alcanzar O(n) en el peor caso. La mediana de tres modifica la selección del pivote, pero no cambia las cotas asintóticas del peor caso.

Tabla 1. Perfil teórico e implementación de los algoritmos comparados.

Algoritmo	Mejor caso	Promedio	Peor caso	Memoria	Estable
Inserción	O(n)	O(n²)	O(n²)	O(1)	Sí
Quicksort (Hoare, mediana de tres)	O(n log n)	O(n log n)	O(n²)	O(log n) promedio; O(n) peor	No

Instancias y protocolo

El generador creó enteros aleatorios uniformes en el rango [0, 1 000 000], sus variantes ordenadas ascendente y descendente, instancias casi ordenadas con el 5 % de las posiciones seleccionadas para permutación, e instancias con duplicados tomadas de 15 valores posibles. Se usó el generador pseudoaleatorio mt19937. El tamaño n tomó los valores 1 000 y 10 000; se repitió cada combinación de tamaño, patrón y algoritmo 30 veces, para 600 observaciones cronometradas. La semilla efectiva se derivó de una semilla base y de la combinación de tamaño, patrón y repetición. Los dos algoritmos recibieron copias de la misma instancia.

La corrida cronometrada no incluyó la generación de entradas, el cálculo de métricas ni la escritura del CSV. El tiempo se midió con std::chrono::steady_clock. Tras detener el reloj, se verificó que el resultado estuviera ordenado. Una segunda corrida sobre una copia idéntica contó comparaciones e intercambios; Quicksort también registró la profundidad máxima de recursión. Para inserción se cuentan los intercambios adyacentes; para Quicksort, los intercambios de elementos realizados durante la partición.

Entorno de ejecución

Las mediciones se ejecutaron en Windows, compilador g++ 14.2.0 de MSYS2 UCRT64, con las opciones -std=c++17 -O2. El procesador identificado fue AMD Ryzen 5 7530U (6 núcleos, 12 hilos, frecuencia base de 2,0 GHz) y el equipo disponía de 16 GB de memoria física. AMD especifica estas características para el modelo del procesador (Advanced Micro Devices, s. f.). La versión del sistema reportó Windows NT 10.0.26200. Se usó el mismo equipo y configuración para todas las condiciones; no se registró la carga de fondo del sistema durante cada repetición.

Resultados

Se conservaron 600 filas en el archivo bruto: 30 repeticiones por algoritmo, patrón y tamaño. El archivo de resumen calcula media, mediana, desviación estándar muestral, mínimo y máximo por condición. La tabla 2 presenta el tiempo medio y su desviación estándar para n = 10 000, además de la razón entre las medias. Una razón mayor que 1 favorece a Quicksort; menor que 1 favorece a inserción.

Tabla 2. Tiempo de ejecución en milisegundos a n = 10 000; media ± desviación estándar (30 repeticiones).

Patrón	Inserción	Quicksort	Inserción / Quicksort
Aleatorio	359.419 ± 6.691	1.196 ± 0.070	300.48×
Ascendente	0.032 ± 0.007	0.279 ± 0.058	0.12×
Descendente	733.264 ± 52.649	0.284 ± 0.060	2584.74×
Casi ordenado	23.879 ± 1.134	0.336 ± 0.078	70.97×
Duplicados	334.595 ± 3.672	0.660 ± 0.087	507.17×

En orden ascendente, inserción promedió 0,032 ms, frente a 0,279 ms de Quicksort. En los otros patrones, Quicksort fue de 71 a 2 584 veces más rápido según la razón de medias. El mayor tiempo de inserción se observó en orden descendente (733,264 ms), seguido por entradas aleatorias (359,419 ms) y con duplicados (334,595 ms). Quicksort registró su mayor tiempo medio en el patrón aleatorio (1,196 ms) y su menor tiempo en el ascendente (0,279 ms).

Las figuras 1 y 2 separan las escalas de los algoritmos para hacer visibles las diferencias entre patrones. La desviación estándar de Quicksort fue mayor en relación con su media en la condición ascendente y en casi ordenado; los tiempos absolutos, sin embargo, siguieron por debajo de 0,31 ms en esas condiciones.

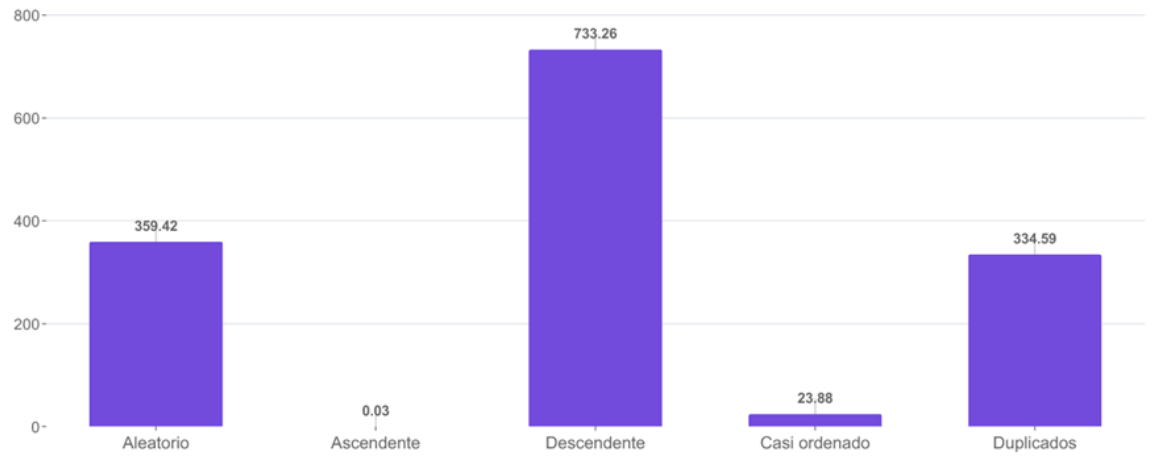


Figura 1. Tiempo promedio de inserción por patrón a n = 10 000; promedio de 30 corridas.

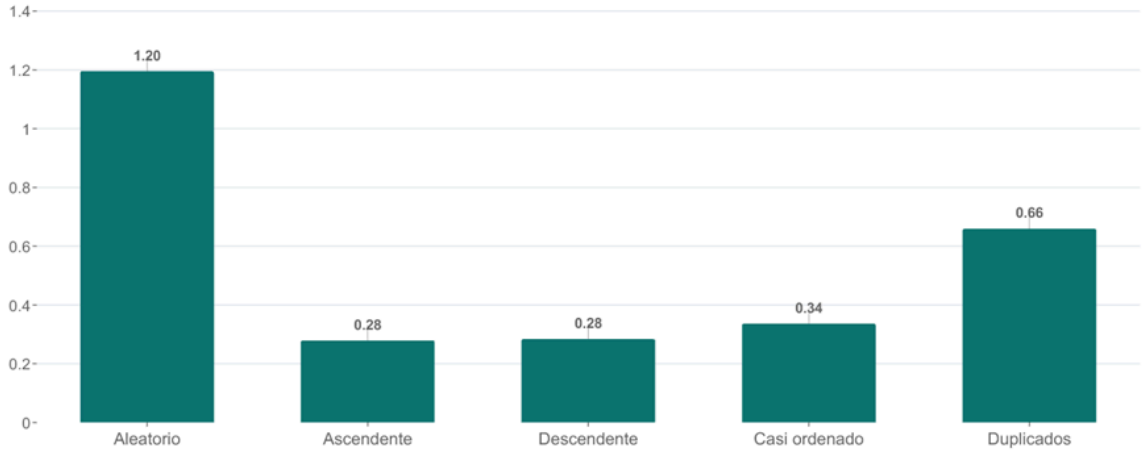


Figura 2. Tiempo promedio de Quicksort por patrón a n = 10 000; promedio de 30 corridas.

Tabla 3. Operaciones medias y profundidad máxima registrada a n = 10 000.

Algoritmo	Patrón	Comparaciones	Intercambios	Profundidad
Inserción	Aleatorio	25,051,482	25,041,491	NA
Inserción	Ascendente	9,999	0	NA
Inserción	Descendente	49,995,000	49,994,950	NA
Inserción	Casi ordenado	1,648,928	1,638,929	NA
Inserción	Duplicados	23,368,594	23,358,597	NA
Quicksort	Aleatorio	198,127	31,768	30
Quicksort	Ascendente	163,665	50	14
Quicksort	Descendente	163,666	5,050	14
Quicksort	Casi ordenado	166,008	1,153	19
Quicksort	Duplicados	188,283	54,522	23

Discusión

Los resultados son consistentes con la sensibilidad de inserción al orden previo. En las entradas ascendentes, cada elemento necesitó una comparación y ningún intercambio, lo que coincide con su mejor caso lineal. En sentido descendente, el promedio fue cercano a $n(n-1)/2$ intercambios, comportamiento propio del peor caso cuadrático. El patrón casi ordenado redujo el costo frente al aleatorio y al descendente, aunque la generación aleatoria de posiciones dejó un número variable de elementos fuera de orden.

Quicksort mostró una menor escala de tiempo en cuatro patrones a n = 10 000. El patrón aleatorio produjo más particiones y trabajo observado que las entradas ascendente y descendente en esta implementación. La distribución con duplicados presentó más intercambios y comparaciones que los patrones ordenados, lo que sugiere que la cantidad de valores repetidos y la partición usada influyen en el costo práctico. Estas mediciones describen una implementación concreta: no deben generalizarse a todas las variantes de Quicksort ni a todos los equipos.

La comparación se limita a n = 10 000 como máximo porque inserción tiene costo cuadrático y una corrida con n = 100 000 resultaría muy prolongada, especialmente para datos aleatorios o descendentes. No se realizó inferencia estadística ni comparación entre varias máquinas. Las 30 repeticiones permiten describir variabilidad dentro de esta sesión, pero no separan cambios de temperatura, carga externa o variaciones entre días. El generador usa mt19937 con reducción por módulo, por lo que la reproducción se garantiza con el mismo código y compilación, aunque no se afirma uniformidad perfecta del mapeo entero.

Conclusiones

1. A n = 10 000, inserción fue claramente más rápida en entradas ascendentes; el orden previo puede reducir su costo de cuadrático a lineal.
2. Quicksort fue más rápido en cuatro de los cinco patrones y mantuvo tiempos medios por debajo de 1,20 ms en las condiciones medidas.
3. Las semillas por entrada, el CSV bruto y el resumen estadístico permiten reproducir las observaciones y relacionarlas con el commit 94f6323.

Referencias

Advanced Micro Devices. (s. f.). AMD Ryzen™ 5 7530U: especificaciones del producto. https://www.amd.com/en/products/processors/laptop/ryzen/7000-series/amd-ryzen-5-7530u.html

Cormen, T. H., Leiserson, C. E., Rivest, R. L., y Stein, C. (2022). Introduction to algorithms (4.ª ed.). MIT Press.

Hoare, C. A. R. (1962). Quicksort. The Computer Journal, 5(1), 10–16. https://doi.org/10.1093/comjnl/5.1.10

Sedgewick, R. (1978). Implementing Quicksort programs. Communications of the ACM, 21(10), 847–857. https://doi.org/10.1145/359619.359631